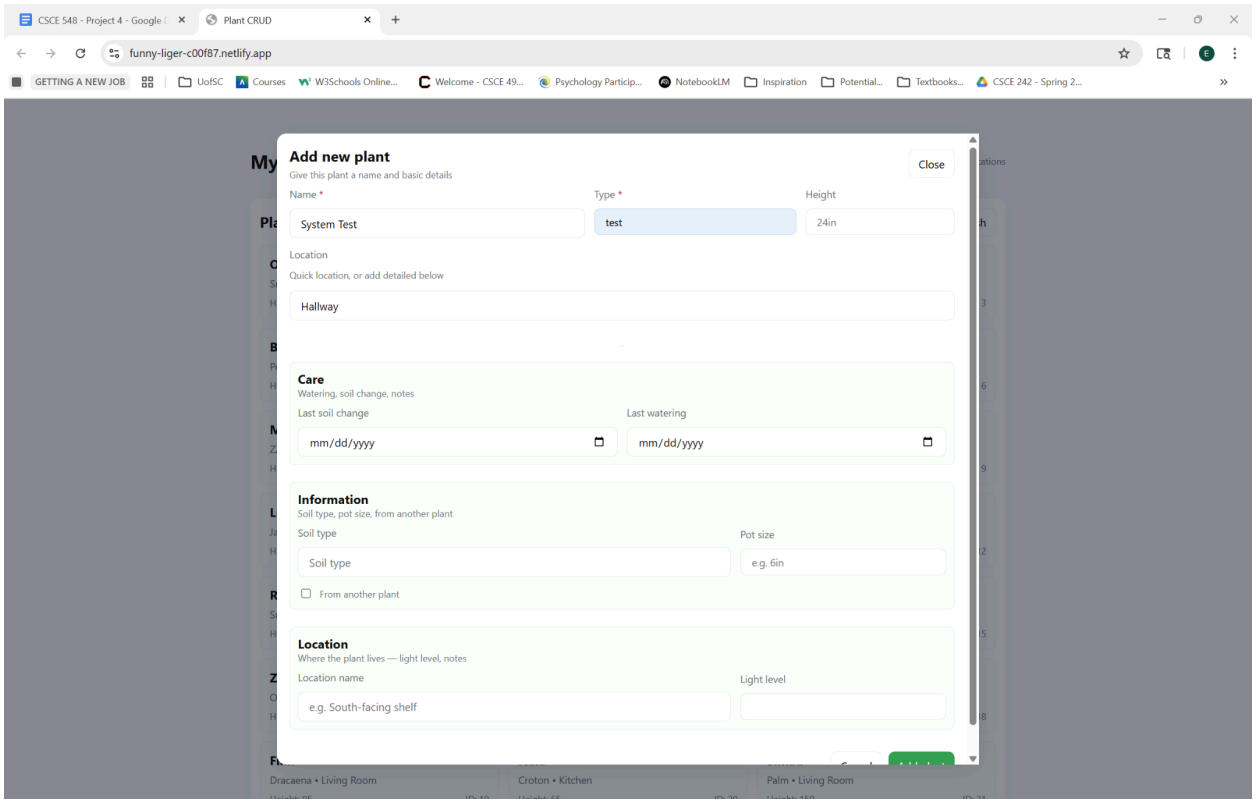
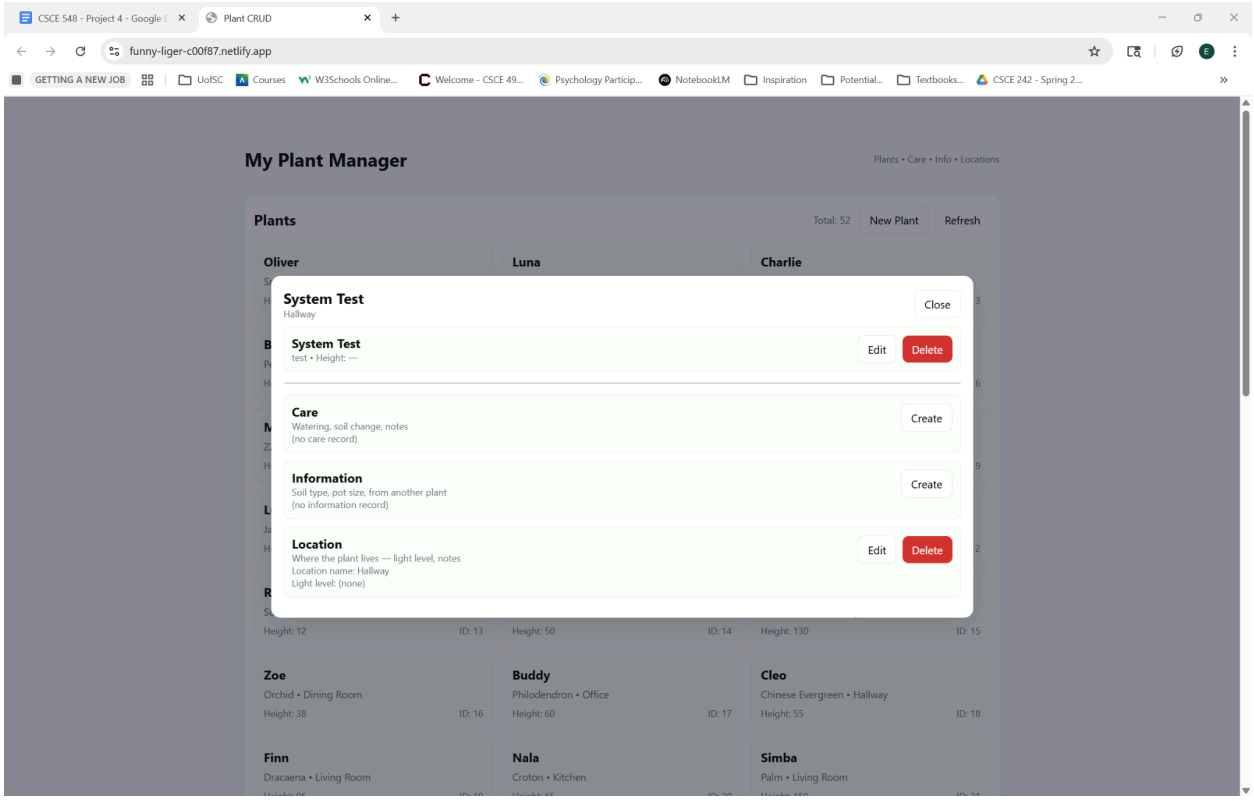
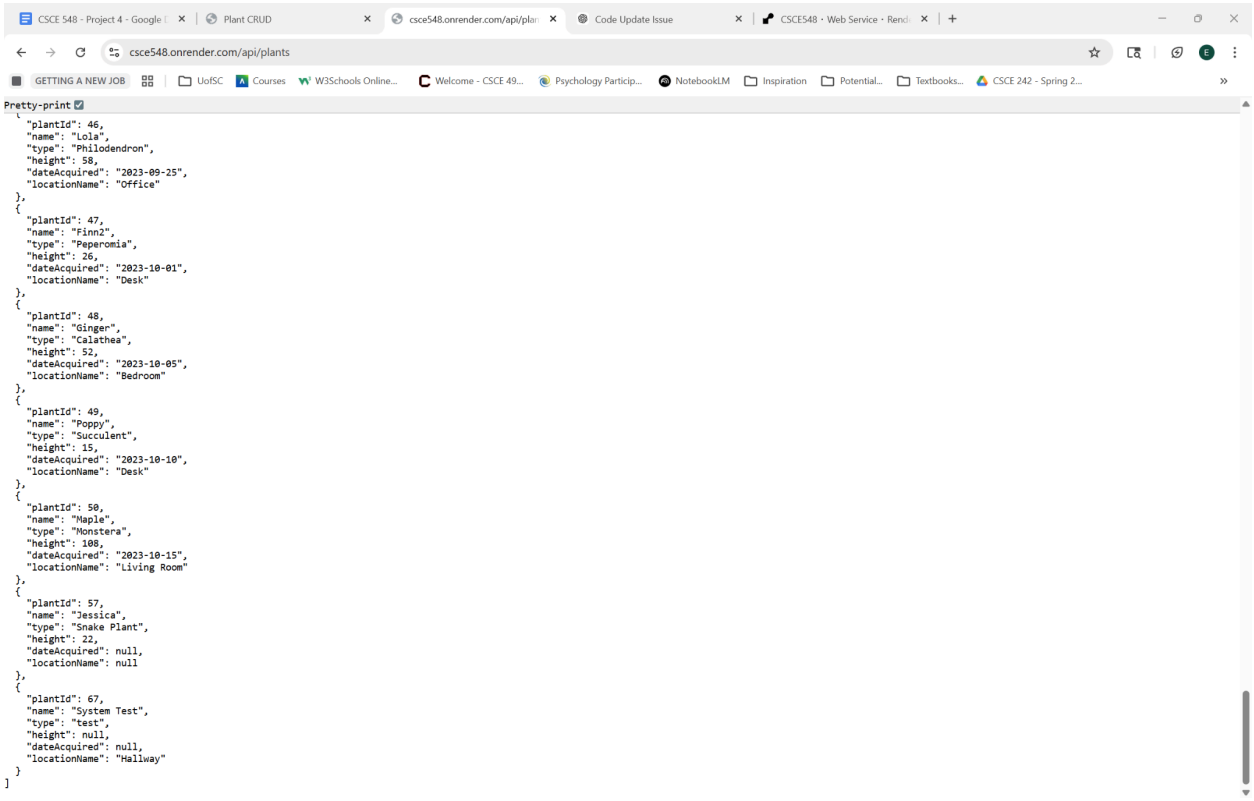
System Test

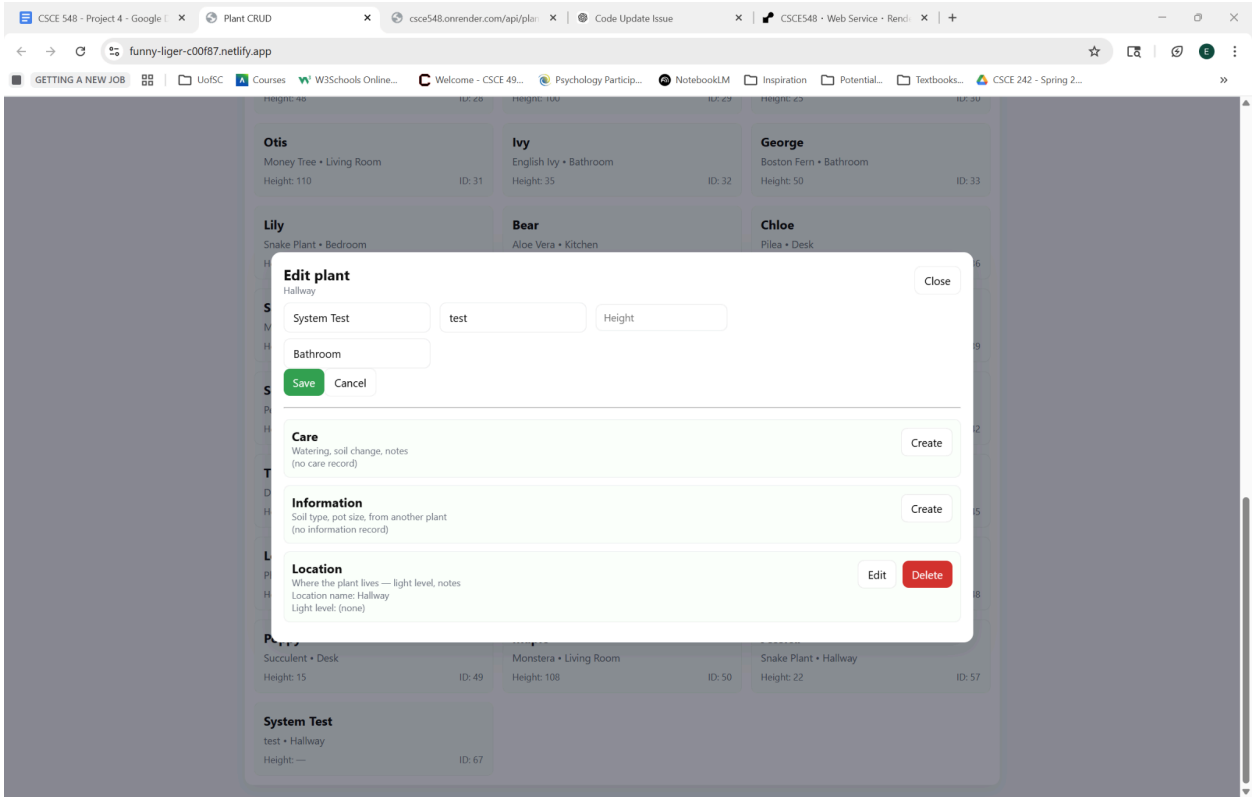
Add functionality:

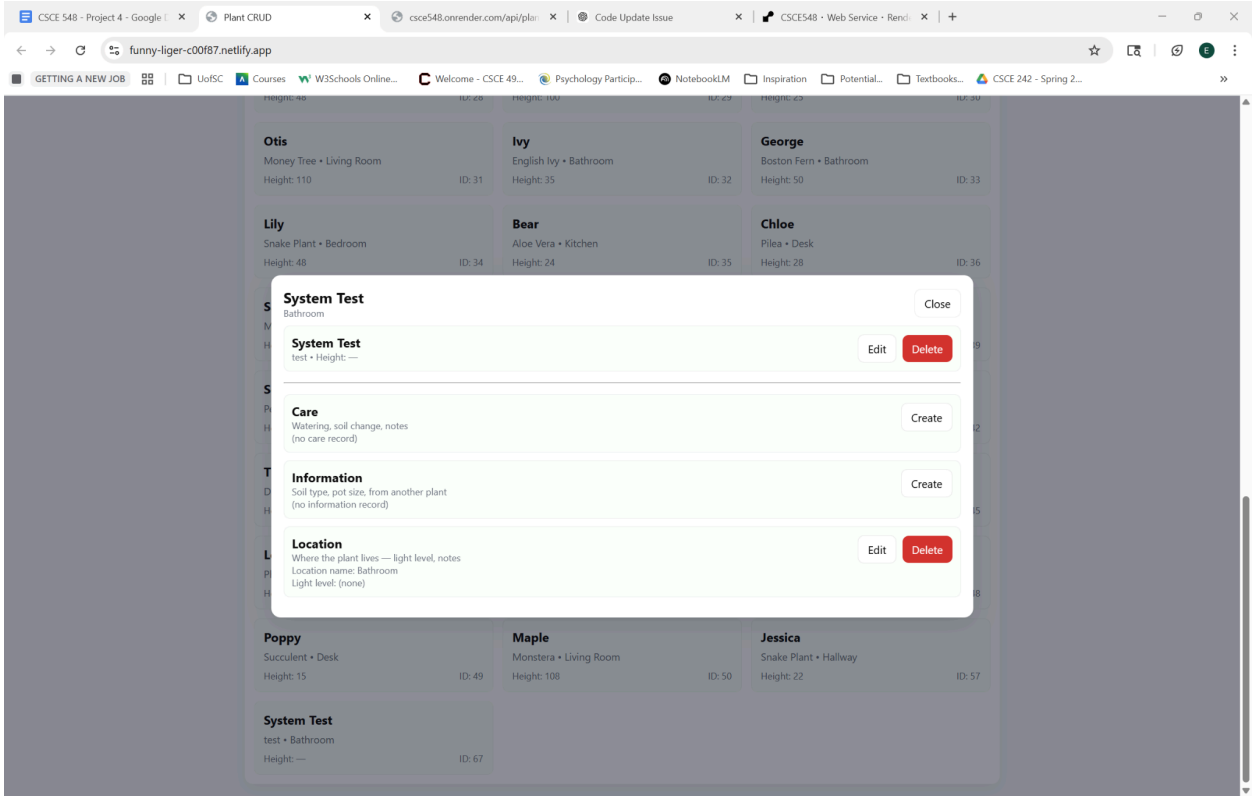


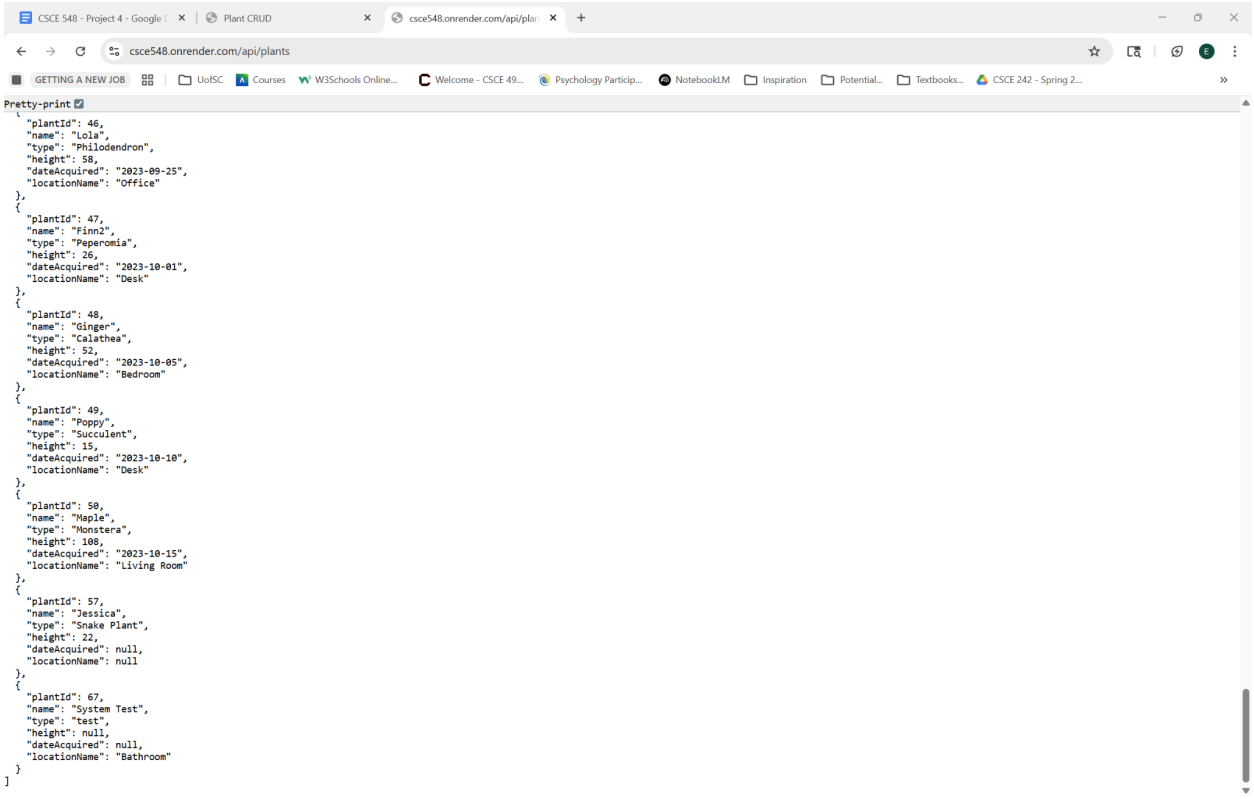

CSCE 548 - Project 4 - Google
Plant CRUD
funny-liger-c00f87.netlify.app
My Plant Manager
Plants • Care • Info • Locations
Plants
Total: 52
New Plant
Refresh
Oliver
Luna
Charlie
System Test
Hallway
Close
System Test
test • Height: —
Edit
Delete
Care
Watering, soil change, notes
(no care record)
Create
Information
Soil type, pot size, from another plant
(no information record)
Create
Location
Where the plant lives — light level, notes
Location name: Hallway
Light level: (none)
Edit
Delete
Height: 12
ID: 13
Height: 50
ID: 14
Height: 130
ID: 15
Zoe
Orchid • Dining Room
Height: 38
ID: 16
Buddy
Philodendron • Office
Height: 60
ID: 17
Cleo
Chinese Evergreen • Hallway
Height: 55
ID: 18
Finn
Dracaena • Living Room
Nala
Croton • Kitchen
Simba
Palm • Living Room


CSCE 548 - Project 4 - Google
Plant CRUD
csce548.onrender.com/api/plan
Code Update Issue
CSCE548 · Web Service · Rend
csce548.onrender.com/api/plants
Pretty-print
[
{
"plantId": 46,
"name": "Lola",
"type": "Philodendron",
"height": 58,
"dateAcquired": "2023-09-25",
"locationName": "Office"
},
{
"plantId": 47,
"name": "Finn2",
"type": "Peperomia",
"height": 26,
"dateAcquired": "2023-10-01",
"locationName": "Desk"
},
{
"plantId": 48,
"name": "Ginger",
"type": "Calathea",
"height": 52,
"dateAcquired": "2023-10-05",
"locationName": "Bedroom"
},
{
"plantId": 49,
"name": "Poppy",
"type": "Succulent",
"height": 15,
"dateAcquired": "2023-10-10",
"locationName": "Desk"
},
{
"plantId": 50,
"name": "Maple",
"type": "Monstera",
"height": 108,
"dateAcquired": "2023-10-15",
"locationName": "Living Room"
},
{
"plantId": 57,
"name": "Jessica",
"type": "Snake Plant",
"height": 22,
"dateAcquired": null,
"locationName": null
},
{
"plantId": 67,
"name": "System Test",
"type": "test",
"height": null,
"dateAcquired": null,
"locationName": "Hallway"
}
]

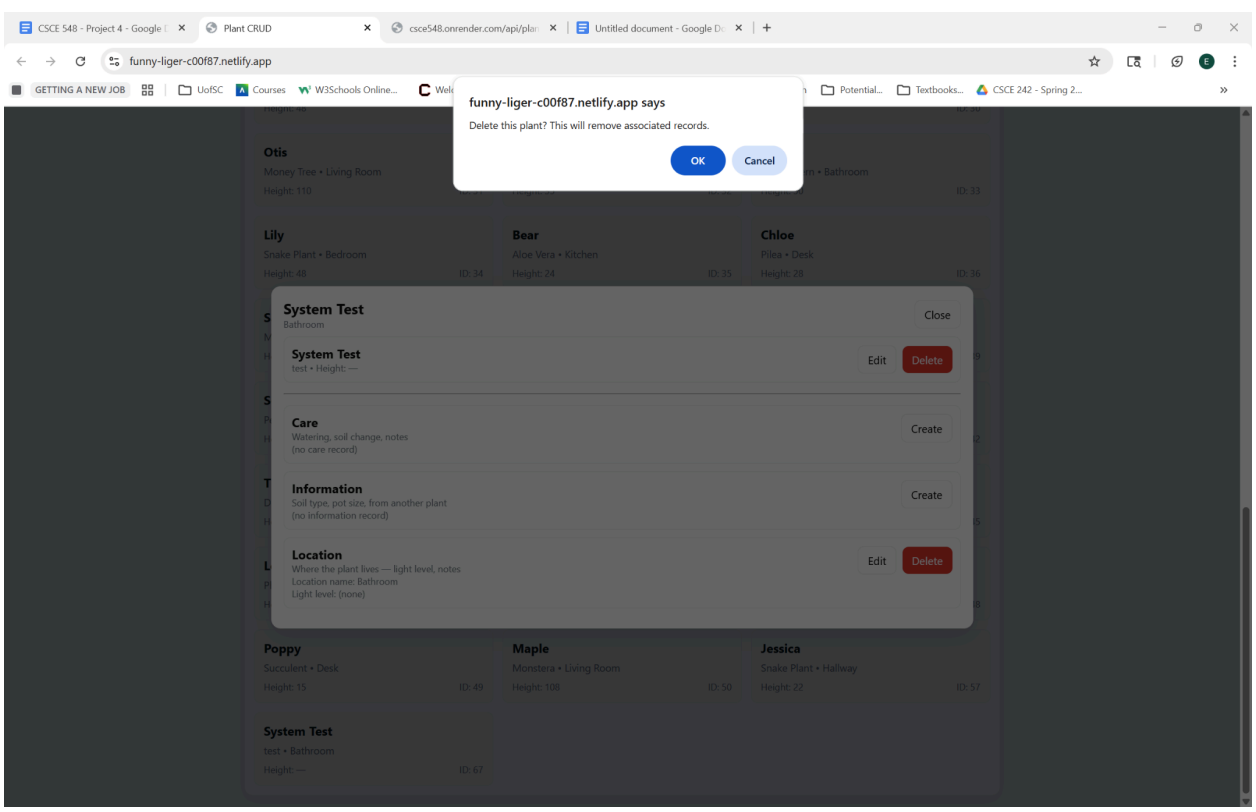
Edit (edited location)

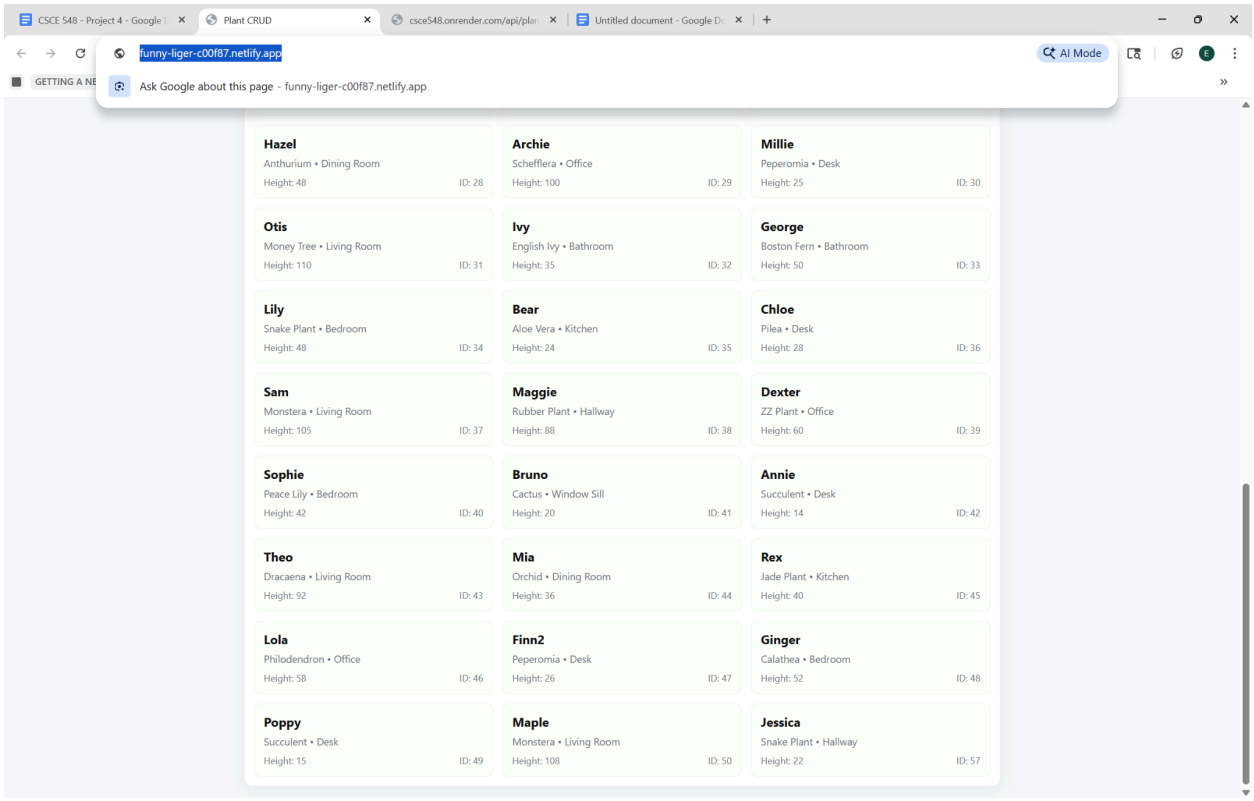


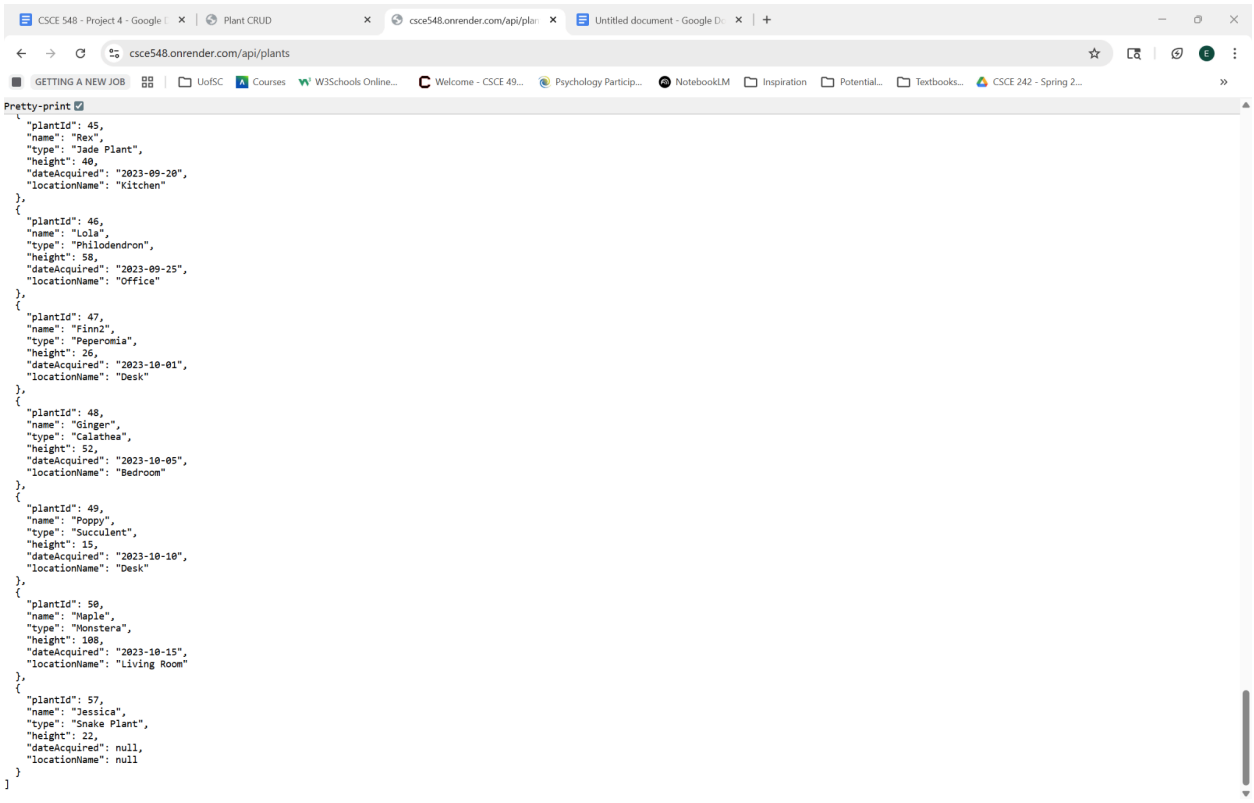



CSCE 548 - Project 4 - Google
Plant CRUD
csce548.onrender.com/api/plan
csce548.onrender.com/api/plants
GETTING A NEW JOB
UofSC
Courses
W3Schools Online...
Welcome - CSCE 49...
Psychology Particip...
NotebookLM
Inspiration
Potential...
Textbooks...
CSCE 242 - Spring 2...
Pretty-print
[
  {
    "plantId": 46,
    "name": "Lola",
    "type": "Philodendron",
    "height": 58,
    "dateAcquired": "2023-09-25",
    "locationName": "Office"
  },
  {
    "plantId": 47,
    "name": "Finn2",
    "type": "Peperomia",
    "height": 26,
    "dateAcquired": "2023-10-01",
    "locationName": "Desk"
  },
  {
    "plantId": 48,
    "name": "Ginger",
    "type": "Calathea",
    "height": 52,
    "dateAcquired": "2023-10-05",
    "locationName": "Bedroom"
  },
  {
    "plantId": 49,
    "name": "Poppy",
    "type": "Succulent",
    "height": 15,
    "dateAcquired": "2023-10-10",
    "locationName": "Desk"
  },
  {
    "plantId": 50,
    "name": "Maple",
    "type": "Monstera",
    "height": 108,
    "dateAcquired": "2023-10-15",
    "locationName": "Living Room"
  },
  {
    "plantId": 57,
    "name": "Jessica",
    "type": "Snake Plant",
    "height": 22,
    "dateAcquired": null,
    "locationName": null
  },
  {
    "plantId": 67,
    "name": "System Test",
    "type": "test",
    "height": null,
    "dateAcquired": null,
    "locationName": "Bathroom"
  }
]

Delete


CSCE 548 - Project 4 - Google
Plant CRUD
csce548.onrender.com/api/plan
Untitled document - Google Do
funny-liger-c00f87.netlify.app
GETTING A NEW JOB
UofSC
Courses
W3Schools Online...
Potential...
Textbooks...
CSCE 242 - Spring 2...
funny-liger-c00f87.netlify.app says
Delete this plant? This will remove associated records.
OK
Cancel
Otis
Money Tree • Living Room
Height: 110
ID: 33
Lily
Snake Plant • Bedroom
Height: 48
ID: 34
Bear
Aloe Vera • Kitchen
Height: 24
ID: 35
Chloe
Pilea • Desk
Height: 28
ID: 36
System Test
Bathroom
Close
System Test
test • Height: —
Edit
Delete
Care
Watering, soil change, notes
(no care record)
Create
Information
Soil type, pot size, from another plant
(no information record)
Create
Location
Where the plant lives — light level, notes
Location name: Bathroom
Light level: (none)
Edit
Delete
Poppy
Succulent • Desk
Height: 15
ID: 49
Maple
Monstera • Living Room
Height: 108
ID: 50
Jessica
Snake Plant • Hallway
Height: 22
ID: 57
System Test
test • Bathroom
Height: —
ID: 67





0 — Quick summary (what you'll end up with)

Backend (Spring Boot) running locally and/or on Render, connecting to PostgreSQL on Clever Cloud.

Frontend (React + Vite) running locally and/or deployed (Netlify or Render static).

Ability to open interactive SQL shell (psql) to Clever Cloud.

Verification steps for each layer.

1 — Project file layout (what we expect)

Root: C:\plant-tracker

Key folders:

```
C:\plant-tracker
├─ .vscode
├─ client\        <-- React + Vite (public, src)
├─ lib\
├─ out\com\planttracker\...
├─ sql\           <-- SQL scripts (migrations / seeds)
├─ src\main\java\com\planttracker\...  <-- backend source
└─ target\         <-- Maven build output
```

2 — Prerequisites

Install these on your machine (Windows examples; macOS/Linux equivalents included):

Dev tools

- Git

  Check: git --version

- Java 17 JDK (Eclipse Temurin/OpenJDK)

  Check: java -version

- Maven 3.6+

  Check: mvn -v

- Node.js 18+ & npm

Check: node -v and npm -v

PostgreSQL client (psql)

Windows: install PostgreSQL or use WSL; ensure psql in PATH.
(we will use SQL Shell)

macOS: brew install postgresql

Ubuntu/Debian: sudo apt install postgresql-client

Optional: Docker, Postman/Insomnia, DBeaver/pgAdmin GUI
(We do have a docker file, but that is specifically for render, we do not need to test it for here)

Accounts

Clever Cloud account with PostgreSQL add-on provisioned (external/direct host preferred).

Render account (for backend) and Netlify account (optional for frontend).

GitHub (push repo to connect to Render/Netlify).

3 — Clone / download the repo

PowerShell:

```
cd C:\
git clone https://github.com/<your-org>/<repo-name>.git plant-tracker
cd plant-tracker
```

Or download ZIP and extract to C:\CSCE548.
cd plant-tracker

Now verify you see client/, src/main/java/..., sql/, target/.
If you do not see those files, try again.

4 — Set up sql server (This will be hosted via clever cloud)
Create an account/Join with github

From there, click on personal space

Then click on Create, an add-on

Select PostgreSQL

Name your database whatever, I used planttracker-dbname

Once created, you will than need to click on the database and collect the following:

Host (external/direct host) e.g. bhnjibolguuj9qrqwpr6-postgresql.services.clever-cloud.com

Port (e.g. 50013)

DB name (e.g. bhnjibolguuj9qrqwpr6)

User (e.g. uh8t1i1ozprgavi7ezlr)

Password (the secret password Clever Cloud gave you)

5 — Use the SQL shell (psql) — interactive prompts

Depending on how you run/connect to the database:
If using simple psql one line:
psql -h Host -p port -U User -d Password

Otherwise enter in the information collected from Clever Cloud via components

From there, enter in the SQL statements (located in the sql folder)

First create the table:

```
psql -U postgres -c "CREATE DATABASE plantdb;"

CREATE TABLE IF NOT EXISTS Plant (
Plant_ID      SERIAL PRIMARY KEY,
Name          VARCHAR(100) NOT NULL,
Type          VARCHAR(100) NOT NULL,
Height        DECIMAL(6,2),
DateAcquired  DATE,
location_name VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS Care (
Plant_ID       INT NOT NULL,
LastSoilChange DATE,
LastWatering   DATE,
PRIMARY KEY (Plant_ID),
CONSTRAINT fk_care_plant
   FOREIGN KEY (Plant_ID)
```

```
    REFERENCES Plant(Plant_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Information (
Plant_ID           INT NOT NULL,
FromAnotherPlant    BOOLEAN DEFAULT FALSE,
SoilType           VARCHAR(100),
PotSize            VARCHAR(50),
WaterGlobeRequired  BOOLEAN DEFAULT FALSE,
PRIMARY KEY (Plant_ID),
CONSTRAINT fk_information_plant
    FOREIGN KEY (Plant_ID)
    REFERENCES Plant(Plant_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Location (
Plant_ID      INT NOT NULL,
location_name VARCHAR(100) NOT NULL,
LightLevel    VARCHAR(50),
PRIMARY KEY (Plant_ID, location_name),
CONSTRAINT fk_location_plant
    FOREIGN KEY (Plant_ID)
    REFERENCES Plant(Plant_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Than fill the database (note, do not paste in the instructions *the parts with the --*):

```
-- If using psql: switch to DB (psql only). If you already connected, skip this.
    \c plantdb

-- 1) Wipe existing data and reset sequences (will remove rows from Plant and all FK children)
    TRUNCATE TABLE plant RESTART IDENTITY CASCADE;

-- 2) Re-insert plants WITHOUT explicit Plant_IDs (Postgres will assign ids 1..50)
    INSERT INTO plant (name, type, height, dateacquired, location_name) VALUES
    ('Oliver', 'Snake Plant', 45.0, '2023-01-10', 'Living Room'),
    ('Luna', 'Pothos', 30.5, '2023-01-15', 'Kitchen'),
    ('Charlie', 'Spider Plant', 28.0, '2023-02-01', 'Bathroom'),
```

('Bella', 'Peace Lily', 40.0, '2023-02-10', 'Bedroom'),
('Max', 'Aloe Vera', 22.0, '2023-02-18', 'Window Sill'),
('Daisy', 'Fiddle Leaf Fig', 120.0, '2023-03-01', 'Living Room'),
('Milo', 'ZZ Plant', 55.0, '2023-03-05', 'Office'),
('Ruby', 'Rubber Plant', 90.0, '2023-03-12', 'Hallway'),
('Oscar', 'Monstera', 110.0, '2023-03-20', 'Living Room'),
('Lucy', 'Jade Plant', 35.0, '2023-03-25', 'Kitchen'),
('Leo', 'Cactus', 18.0, '2023-04-01', 'Window Sill'),
('Molly', 'Fern', 42.0, '2023-04-05', 'Bathroom'),
('Rocky', 'Succulent', 12.0, '2023-04-10', 'Desk'),
('Willow', 'Calathea', 50.0, '2023-04-15', 'Bedroom'),
('Toby', 'Bamboo Palm', 130.0, '2023-04-20', 'Living Room'),
('Zoe', 'Orchid', 38.0, '2023-04-25', 'Dining Room'),
('Buddy', 'Philodendron', 60.0, '2023-05-01', 'Office'),
('Cleo', 'Chinese Evergreen', 55.0, '2023-05-05', 'Hallway'),
('Finn', 'Dracaena', 95.0, '2023-05-10', 'Living Room'),
('Nala', 'Croton', 65.0, '2023-05-15', 'Kitchen'),
('Simba', 'Palm', 150.0, '2023-05-20', 'Living Room'),
('Penny', 'Begonia', 30.0, '2023-05-25', 'Bedroom'),
('Henry', 'Yucca', 140.0, '2023-06-01', 'Hallway'),
('Rosie', 'African Violet', 20.0, '2023-06-05', 'Window Sill'),
('Jack', 'Dieffenbachia', 85.0, '2023-06-10', 'Office'),
('Ellie', 'Prayer Plant', 45.0, '2023-06-15', 'Bedroom'),
('Louie', 'Parlor Palm', 70.0, '2023-06-20', 'Living Room'),
('Hazel', 'Anthurium', 48.0, '2023-06-25', 'Dining Room'),
('Archie', 'Schefflera', 100.0, '2023-07-01', 'Office'),
('Millie', 'Peperomia', 25.0, '2023-07-05', 'Desk'),
('Otis', 'Money Tree', 110.0, '2023-07-10', 'Living Room'),
('Ivy', 'English Ivy', 35.0, '2023-07-15', 'Bathroom'),
('George', 'Boston Fern', 50.0, '2023-07-20', 'Bathroom'),
('Lily', 'Snake Plant', 48.0, '2023-07-25', 'Bedroom'),
('Bear', 'Aloe Vera', 24.0, '2023-08-01', 'Kitchen'),
('Chloe', 'Pilea', 28.0, '2023-08-05', 'Desk'),
('Sam', 'Monstera', 105.0, '2023-08-10', 'Living Room'),
('Maggie', 'Rubber Plant', 88.0, '2023-08-15', 'Hallway'),
('Dexter', 'ZZ Plant', 60.0, '2023-08-20', 'Office'),
('Sophie', 'Peace Lily', 42.0, '2023-08-25', 'Bedroom'),
('Bruno', 'Cactus', 20.0, '2023-09-01', 'Window Sill'),
('Annie', 'Succulent', 14.0, '2023-09-05', 'Desk'),
('Theo', 'Dracaena', 92.0, '2023-09-10', 'Living Room'),
('Mia', 'Orchid', 36.0, '2023-09-15', 'Dining Room'),
('Rex', 'Jade Plant', 40.0, '2023-09-20', 'Kitchen'),
('Lola', 'Philodendron', 58.0, '2023-09-25', 'Office'),
('Finn2', 'Peperomia', 26.0, '2023-10-01', 'Desk'),

```
('Ginger', 'Calathea', 52.0, '2023-10-05', 'Bedroom'),
('Poppy', 'Succulent', 15.0, '2023-10-10', 'Desk'),
('Maple', 'Monstera', 108.0, '2023-10-15', 'Living Room');

-- 3) Insert Care rows referencing plant IDs 1..50 (these IDs will match because we reset the sequence)
INSERT INTO care (plant_id, lastsoilchange, lastwatering) VALUES
(1, '2023-09-01','2026-01-21'),
(2, '2023-09-02','2026-01-21'),
(3, '2023-09-03','2026-01-21'),
(4, '2023-09-04','2026-01-21'),
(5, '2023-09-05','2026-01-21'),
(6, '2023-09-06','2026-01-21'),
(7, '2023-09-07','2026-01-21'),
(8, '2023-09-08','2026-01-21'),
(9, '2023-09-09','2026-01-21'),
(10, '2023-09-10','2026-01-21'),
(11, '2023-09-11','2026-01-21'),
(12, '2023-09-12','2026-01-21'),
(13, '2023-09-13','2026-01-21'),
(14, '2023-09-14','2026-01-21'),
(15, '2023-09-15','2026-01-21'),
(16, '2023-09-16','2026-01-21'),
(17, '2023-09-17','2026-01-21'),
(18, '2023-09-18','2026-01-21'),
(19, '2023-09-19','2026-01-21'),
(20, '2023-09-20','2026-01-21'),
(21, '2023-09-21','2026-01-21'),
(22, '2023-09-22','2026-01-21'),
(23, '2023-09-23','2026-01-21'),
(24, '2023-09-24','2026-01-21'),
(25, '2023-09-25','2026-01-21'),
(26, '2023-09-26','2026-01-21'),
(27, '2023-09-27','2026-01-21'),
(28, '2023-09-28','2026-01-21'),
(29, '2023-09-29','2026-01-21'),
(30, '2023-09-30','2026-01-21'),
(31, '2023-10-01','2026-01-21'),
(32, '2023-10-02','2026-01-21'),
(33, '2023-10-03','2026-01-21'),
(34, '2023-10-04','2026-01-21'),
(35, '2023-10-05','2026-01-21'),
(36, '2023-10-06','2026-01-21'),
(37, '2023-10-07','2026-01-21'),
```

```
(38, '2023-10-08','2026-01-21'),
(39, '2023-10-09','2026-01-21'),
(40, '2023-10-10','2026-01-21'),
(41, '2023-10-11','2026-01-21'),
(42, '2023-10-12','2026-01-21'),
(43, '2023-10-13','2026-01-21'),
(44, '2023-10-14','2026-01-21'),
(45, '2023-10-15','2026-01-21'),
(46, '2023-10-16','2026-01-21'),
(47, '2023-10-17','2026-01-21'),
(48, '2023-10-18','2026-01-21'),
(49, '2023-10-19','2026-01-21'),
(50, '2023-10-20','2026-01-21');

-- 4) Insert Information rows
INSERT INTO information (plant_id, fromanotherplant, soiltype, potsize,
watergloberequired) VALUES
(1, FALSE, 'All-purpose potting soil', '8 inch', FALSE),
(2, FALSE, 'All-purpose potting soil', '6 inch', FALSE),
(3, TRUE,  'Peat-based mix', '6 inch', FALSE),
(4, FALSE, 'Moisture-retentive mix', '8 inch', TRUE),
(5, TRUE,  'Cactus & succulent mix', '4 inch', FALSE),
(6, FALSE, 'Indoor tree mix', '12 inch', TRUE),
(7, FALSE, 'All-purpose potting soil', '8 inch', FALSE),
(8, FALSE, 'Indoor tree mix', '10 inch', FALSE),
(9, TRUE,  'Aroid mix', '12 inch', TRUE),
(10, FALSE,'Succulent mix', '6 inch', FALSE),
(11, FALSE,'Cactus & succulent mix', '3 inch', FALSE),
(12, TRUE, 'Fern mix', '8 inch', TRUE),
(13, FALSE,'Succulent mix', '3 inch', FALSE),
(14, FALSE,'Moisture-retentive mix', '6 inch', TRUE),
(15, FALSE,'Palm/large plant mix', '14 inch', TRUE),
(16, FALSE,'Orchid bark mix', '4 inch', TRUE),
(17, TRUE, 'All-purpose potting soil', '8 inch', FALSE),
(18, FALSE,'All-purpose potting soil', '8 inch', FALSE),
(19, FALSE,'Indoor tree mix', '12 inch', TRUE),
(20, FALSE,'Well-draining mix', '8 inch', FALSE),
(21, FALSE,'Palm/large plant mix', '16 inch', TRUE),
(22, TRUE, 'Begonia mix', '6 inch', FALSE),
(23, FALSE,'Sandy well-draining mix', '14 inch', TRUE),
(24, FALSE,'African violet mix', '3 inch', FALSE),
(25, TRUE, 'All-purpose potting soil', '10 inch', FALSE),
(26, FALSE,'Moisture-retentive mix', '6 inch', TRUE),
(27, FALSE,'Palm/large plant mix', '10 inch', TRUE),
```

```
(28, FALSE,'Aroid mix', '8 inch', TRUE),
(29, TRUE, 'All-purpose potting soil', '12 inch', FALSE),
(30, FALSE,'Succulent mix', '4 inch', FALSE),
(31, FALSE,'Money tree mix', '12 inch', TRUE),
(32, TRUE, 'Ivy mix', '6 inch', FALSE),
(33, FALSE,'Fern mix', '8 inch', TRUE),
(34, FALSE,'All-purpose potting soil', '8 inch', FALSE),
(35, TRUE, 'Cactus & succulent mix', '4 inch', FALSE),
(36, FALSE,'Peat-based mix', '6 inch', FALSE),
(37, FALSE,'Aroid mix', '12 inch', TRUE),
(38, FALSE,'Indoor tree mix', '10 inch', FALSE),
(39, TRUE, 'All-purpose potting soil', '8 inch', FALSE),
(40, FALSE,'Moisture-retentive mix', '8 inch', TRUE),
(41, FALSE,'Cactus & succulent mix', '3 inch', FALSE),
(42, TRUE, 'Succulent mix', '3 inch', FALSE),
(43, FALSE,'Indoor tree mix', '12 inch', TRUE),
(44, FALSE,'Orchid bark mix', '4 inch', TRUE),
(45, TRUE, 'Succulent mix', '6 inch', FALSE),
(46, FALSE,'All-purpose potting soil', '8 inch', FALSE),
(47, FALSE,'Succulent mix', '4 inch', FALSE),
(48, FALSE,'Moisture-retentive mix', '8 inch', TRUE),
(49, TRUE, 'Succulent mix', '3 inch', FALSE),
(50, FALSE,'Aroid mix', '12 inch', TRUE);

-- 5) Insert Location rows
INSERT INTO location (plant_id, location_name, lightlevel) VALUES
(1, 'Living Room', 'Bright'),
(2, 'Kitchen', 'Medium'),
(3, 'Bathroom', 'Low'),
(4, 'Bedroom', 'Low'),
(5, 'Window Sill', 'Bright'),
(6, 'Living Room', 'Bright'),
(7, 'Office', 'Medium'),
(8, 'Hallway', 'Low'),
(9, 'Living Room', 'Bright'),
(10, 'Kitchen', 'Medium'),
(11, 'Window Sill', 'Bright'),
(12, 'Bathroom', 'Low'),
(13, 'Desk', 'Low'),
(14, 'Bedroom', 'Low'),
(15, 'Living Room', 'Bright'),
(16, 'Dining Room', 'Medium'),
(17, 'Office', 'Medium'),
(18, 'Hallway', 'Low'),
```

```
(19, 'Living Room', 'Bright'),
(20, 'Kitchen', 'Medium'),
(21, 'Living Room', 'Bright'),
(22, 'Bedroom', 'Low'),
(23, 'Hallway', 'Low'),
(24, 'Window Sill', 'Bright'),
(25, 'Office', 'Medium'),
(26, 'Bedroom', 'Low'),
(27, 'Living Room', 'Bright'),
(28, 'Dining Room', 'Medium'),
(29, 'Office', 'Medium'),
(30, 'Desk', 'Low'),
(31, 'Living Room', 'Bright'),
(32, 'Bathroom', 'Low'),
(33, 'Bathroom', 'Low'),
(34, 'Bedroom', 'Low'),
(35, 'Kitchen', 'Medium'),
(36, 'Desk', 'Low'),
(37, 'Living Room', 'Bright'),
(38, 'Hallway', 'Low'),
(39, 'Office', 'Medium'),
(40, 'Bedroom', 'Low'),
(41, 'Window Sill', 'Bright'),
(42, 'Desk', 'Low'),
(43, 'Living Room', 'Bright'),
(44, 'Dining Room', 'Medium'),
(45, 'Kitchen', 'Medium'),
(46, 'Office', 'Medium'),
(47, 'Desk', 'Low'),
(48, 'Bedroom', 'Low'),
(49, 'Desk', 'Low'),
(50, 'Living Room', 'Bright');
```

Quick psql commands after connecting

```
\dt              -- list tables
\dn              -- list schemas
SELECT NOW();      -- server time check
SELECT count(*) FROM plants;  -- example
\q               -- quit
```

If you get SSL errors, confirm the host/port and the JDBC sslmode=require requirement. For CLI psql, there’s typically nothing to change; the interactive connection should work with the external host/port.

6 — Important: DbConfig in your Java code (this is strictly for testing the backend)

This DbConfig class — these exact environment variable names must be used (e.g. "PLANTDB_URL"):
We are only changing the information to match our Clever Cloud

```
// DbConfig (reference)
public class DbConfig {
    public static String getJdbcUrl() {
        return System.getenv().getOrDefault(
            "PLANTDB_URL",
            "jdbc:postgresql://host:port/databaseName?sslmode=require"
        );
    }

    public static String getUser() {
        return System.getenv().getOrDefault("PLANTDB_USER", "CLEVER_CLOUD_USER");
    }

    public static String getPassword() {
        return System.getenv().getOrDefault("PLANTDB_PASS",
"CLEVER_CLOUD_PASSWORD");
    }
}
```

Defaults are a Clever Cloud example; set PLANTDB_PASS (never committed).

The JDBC URL must be jdbc:postgresql://... form for JDBC connections.

7 — Backend: compile & run locally
We are checking to see if what we have hosted on Clever Cloud works

For this case, we are using spring-boot

Root of Maven project: C:\plant-tracker (run commands there)

1) Build with Maven
```
cd C:\plant-tracker
mvn clean package
```
2) Run

Run via Maven:

```
mvn spring-boot:run
```

Option B — run packaged jar (this has never worked for me):

```
java -jar target\planttracker-0.0.1-SNAPSHOT.jar
```

App should bind to port 8080 (unless changed). Check console logs for successful DB connection.

For plants: http://localhost:8080/api/plants
For details: http://localhost:8080/api/plants/{id}/care (you need to have an id number 1-whatever the largest number is in the database)

Without the API plants it will give you a 404

8 — Deploy Backend to Render (step-by-step)

Goal: Build the Spring Boot app with Maven on Render. Set the correct environment variables (Clever Cloud DB). Use the repository you pushed to GitHub.

1) Push your repo to GitHub (PowerShell)

```
cd C:\plant-tracker
git init
git add .
git commit -m "deploy-ready"
git branch -M main
git remote add origin https://github.com/<your-org>/<repo-name>.git
git push -u origin main
```

2) Create a Web Service on Render

Sign in to Render and click New → Web Service.

Connect your GitHub account and select the repo.

On render click new -> Web Service

In the Create Web Service form set:

Name: CSCE548 (or your preferred name)

Lanugage: Docker (NOTE, WE NEED THIS TO BE DOCKER)

Environment: Service (default)

Region: pick one near your DB / users

Root Directory: plant-tracker

Branch: main

Dockerfile Path: Dockerfile

Docker Build Context Directory: .

(We don't need to do the Pre-Deploy command because it's aready in the Dockerfile)

Instance Type: choose smallest for testing, scale later

Tip: If your project uses a Dockerfile for Render, choose the Docker option and Render will build via Docker instead; the remainder (env vars) is the same.

3) Add environment variables (Render UI → Environment)

Add these exact names (values from Clever Cloud):

PLANTDB_URL - jdbc:postgresql://<host>:<port>/<dbname>?sslmode=require
Example:
jdbc:postgresql://bhnjibolguuj9qrqwpr6-postgresql.services.clever-cloud.com:50013/bhnjibolguuj9qrqwpr6?sslmode=require

PLANTDB_USER — e.g. uh8t1i1ozprgavi7ezlr

PLANTDB_PASS — the Clever Cloud password (secret — do not commit)

4) Deploy

Click Create Web Service or Manual Deploy → Render will build using the Build Command and run the Start Command.

Watch the Build Logs for mvn progress and the Instance Logs for startup.

5) Verify backend is running

You'll get a Render URL such as https://<your-service>.onrender.com.

Use PowerShell to check:

Invoke-RestMethod -Uri "https://<your-service>.onrender.com/api/plants" -Method Get

Check Render logs for successful DB connection lines (your app's log usually prints something like "Connected to database" or no JDBC errors). If you see JDBC auth, URL, or SSL errors, re-check the env vars.

9 — Frontend: local dev and build

Frontend build environment:

VITE_API_BASE = https://csce548.onrender.com (production *change via whatever render link you got*)

For local dev override: VITE_API_BASE=http://localhost:8080/api

Note: Vite inlines env vars at build time. Ensure Netlify/Render static build has VITE_API_BASE set to the deployed backend URL.

Frontend located at C:\plant-tracker\client.

Local dev

PowerShell:

```
cd C:\plant-tracker\client
npm install
npm install vite
# set API base for dev (session-only)
```

For local host/temp display, run:
npm run dev

preview (recommend doing before publishing)
npm run preview

Build for production
npm run build

Open: http://localhost:5173. Use devtools → Network to watch API calls.

Output: client\dist (publish directory).

Ensure VITE_API_BASE is set to https://csce548.onrender.com before building for production on Netlify/Render.

10 — Deploy Frontend (Netlify) — adjusted for your repo

Goal: Build the React + Vite client and host it on Netlify.
Key rule: Vite inlines env vars at build time — set VITE_API_BASE in Netlify before the site build. Use the Render backend URL you will run in production (for your project: https://csce548.onrender.com or https://csce548.onrender.com/api depending on how the client expects it).

Below is a copy-ready, step-by-step Netlify section tuned to your repo (client lives at C:\plant-tracker\client).

10.1 Prepare the repo / client (local sanity checks)

If your repo root contains client/, set Netlify's Base directory to client (Netlify will run the build there and the Vite output will be client/dist).

Local checks (PowerShell):

```
cd C:\plant-tracker\client

# install reproducible dependencies
npm ci

# optional: set env for local dev session (session-only)
$env:VITE_API_BASE="http://localhost:8080/api"   # for local dev if backend runs on localhost

# run the Vite dev server
npm run dev

# create a production build locally to confirm it works
npm run build

# preview the production build (optional)
npm run preview
# preview serves the dist on a local port so you can confirm static output
```

If preview doesn't start, open client/dist/index.html in a browser to sanity-check static generation (or use npx serve client/dist).

Confirm the app loads and that in DevTools network requests go to the URL in VITE_API_BASE (for npm run dev you used http://localhost:8080/api).

10.2 Commit & push client changes (trigger deploy)

PowerShell:

```
cd C:\plant-tracker
git add client
git commit -m "frontend: ready for netlify"
git push origin main
```

Netlify will build automatically on the first import or when you push to the connected branch, but you must set the env var first or re-trigger a build after changing it (see below).

10.3 Create site on Netlify (web UI steps)

Sign in to Netlify → New site → Import from Git → choose GitHub and pick your repo.

Configure the build:

Branch to deploy: main (or your branch)

Base directory: client
(this makes the build run in client/ and output client/dist)

Build command:
npm ci && npm run build
(or npm run build if you prefer to run npm ci earlier in your pipeline)

Publish directory: dist

Click Deploy site (first build will run — but set env var first if you want it to point to production backend immediately).

10.4 Set VITE_API_BASE in Netlify BEFORE the build

Netlify UI: Site → Site settings → Build & deploy → Environment

Add exact key name:

VITE_API_BASE = https://<render name>.onrender.com\

(Note, you can also just the the .env file to your render link)

10.5 SPA routing note — ensure client-side routing works

If your app uses client-side routes (React Router), create client/static/_redirects or client/public/_redirects with:

```
/*    /index.html   200
```

Netlify will deploy that and ensure all paths are served to index.html (no 404 for client routes). Commit that file before building.

10.6 Trigger a manual deploy (when you change env vars)

Any change to build environment variables requires a rebuild. In Netlify UI:

Site → Deploys → Trigger deploy → Deploy site (or Clear cache and deploy if you changed dependencies/CI).

You can also push a trivial commit to trigger a new build.

10.7 Verify deployed site

Netlify provides a URL like https://<random>-site.netlify.app (or your custom domain).

Open the site in a browser and exercise features that call the API (list plants, create, edit).

In DevTools → Network verify that requests target the Render backend domain you set in VITE_API_BASE (for your setup: https://csce548.onrender.com/api/...).

PowerShell quick check (no curl):

```
# Check site root loads
Invoke-RestMethod -Uri "https://<random-site>.netlify.app" -Method Get

# If you want to test directly against the backend URL that the client uses:
Invoke-RestMethod -Uri "https://csce548.onrender.com/api/plants" -Method Get
```

Perform a full create from the UI, then verify the DB row exists via psql (see the verification section in your doc).

11 — Recommended repo files to add (copy into root)

.env.example (do NOT commit real passwords):

```
# Backend (DbConfig)
PLANTDB_URL=jdbc:postgresql://<host>:<port>/<dbname>?sslmode=require
PLANTDB_USER=<dbuser>
PLANTDB_PASS=<REDACTED>

# Frontend (Vite)
VITE_API_BASE=https://csce548.onrender.com
```